



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +



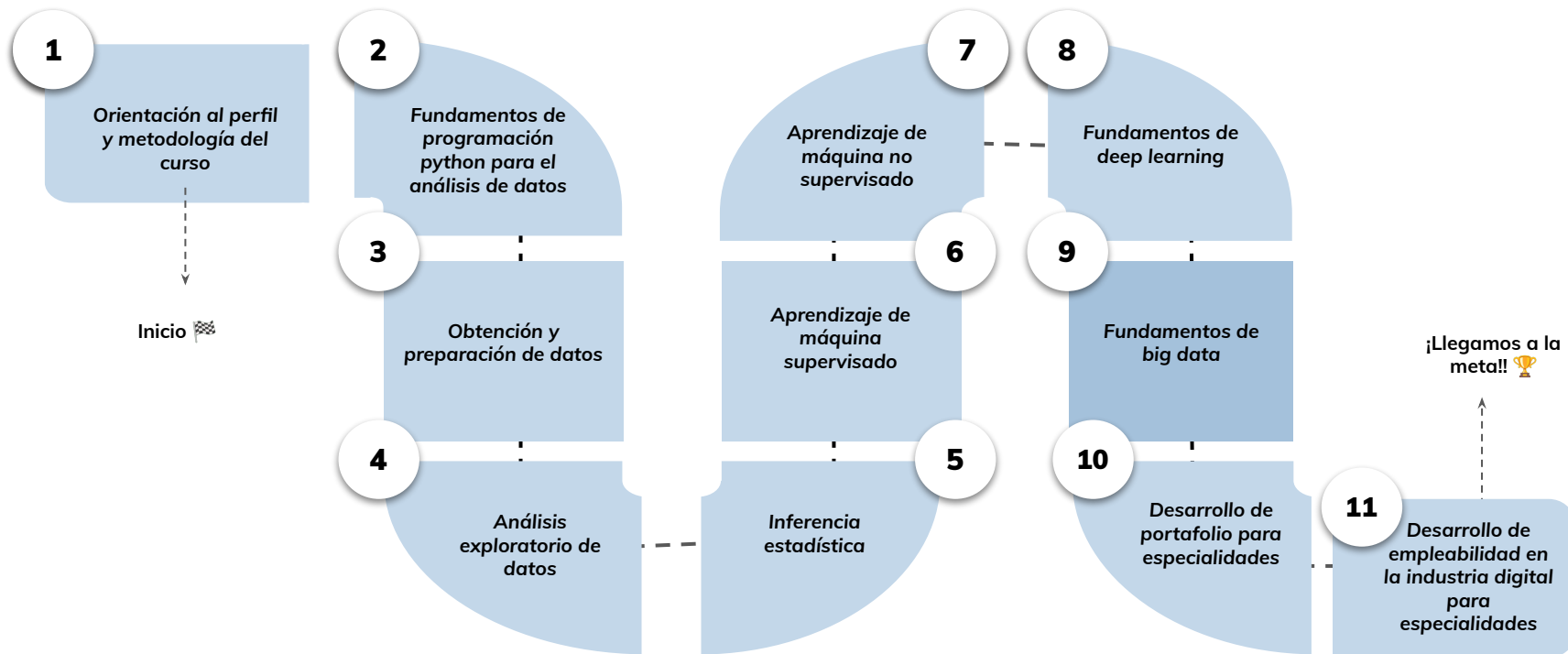
Elementos básicos de spark-

› Parte I

Aprendizaje Esperado 3: Manipular grandes volúmenes de datos utilizando objetos rdd, transformaciones y acciones para resolver un problema en el entorno spark.

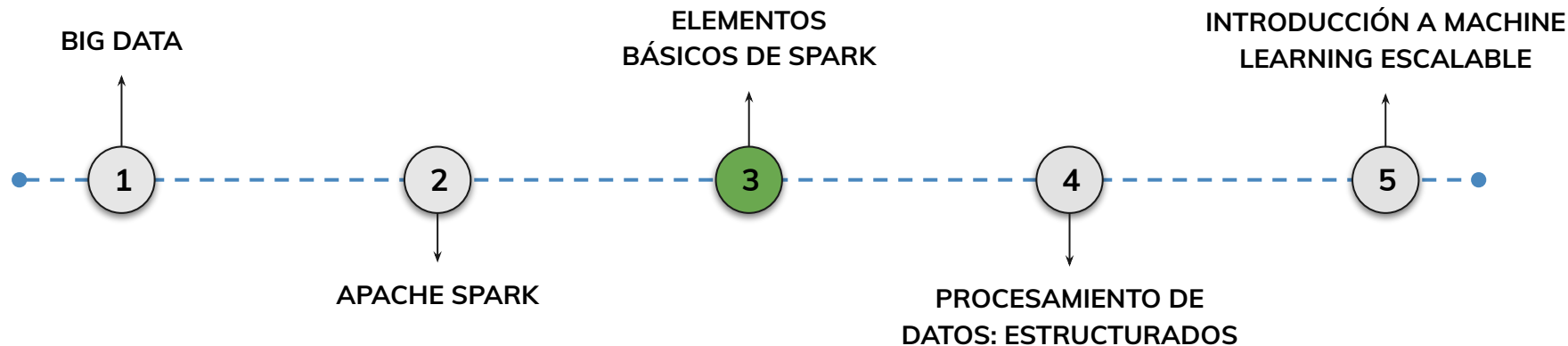
Hoja de ruta

¿Cuáles skills conforman el programa? Fundamentos de Ciencia de Datos



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

3.

Manipulación de datos con
RDD, transformaciones y
acciones en Spark

Aprenderemos a utilizar los elementos básicos de Spark para trabajar con grandes volúmenes de datos, desde la creación de RDD hasta la aplicación de transformaciones y acciones para obtener resultados.

Fundamentos de RDD y su
uso en Spark

Qué es un RDD y cómo se crea.

Persistencia y cacheo de datos.

Transformaciones y
acciones en RDD

Tipos y ejemplos de transformaciones: map, filter, flatMap, reduceByKey, join.

Acciones: collect, take, count, sum, mean.



Objetivos de aprendizaje

¿Qué aprenderás?



- Comprender qué es un RDD y su rol en Spark.
- Crear, persistir y manipular RDDs desde distintas fuentes.
- Aplicar transformaciones para preparar y filtrar datos.
- Ejecutar acciones para obtener resultados concretos.
- Entender el concepto de linaje y lazy evaluation para optimizar procesos.

Repaso clase anterior

¿Quedó alguna duda?

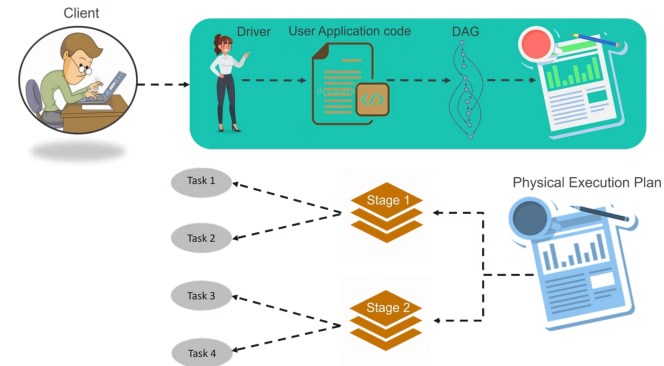
En la clase anterior trabajamos :

- Analizamos la arquitectura distribuida de Apache Spark.
- Identificamos los componentes clave: Driver, Executors y Cluster Manager.
- Entendimos el ciclo de un Job Spark: DAG → stages → tasks.
- Exploramos distintas formas de implementar Spark (Standalone, YARN, Kubernetes, PySpark).

Elementos básicos de spark

Apache Spark: Fundamentos y Procesamiento de Datos Distribuido

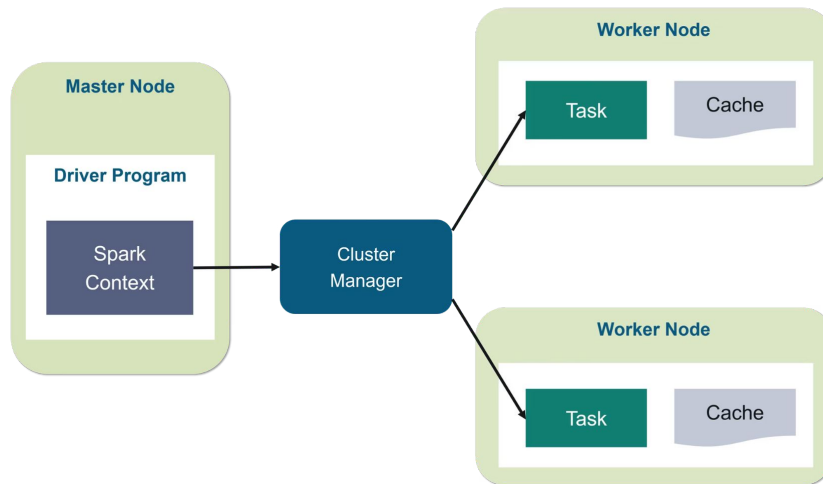
Apache Spark es uno de los motores de procesamiento de datos más potentes y utilizados en la actualidad para manejar grandes volúmenes de información. Su principal característica es la capacidad de realizar análisis masivo de datos de forma distribuida y en memoria, lo que permite obtener resultados de manera más rápida que sistemas tradicionales como Hadoop MapReduce.

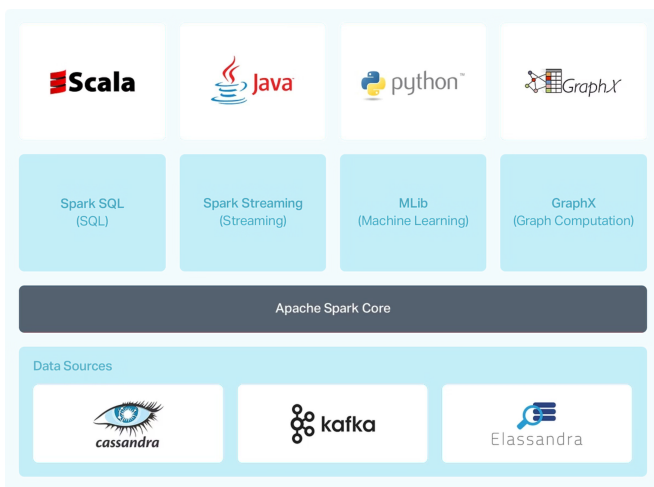


Introducción a Apache Spark

Para trabajar con Spark es fundamental conocer sus componentes básicos, entre los que se destacan los **RDD** (Resilient Distributed Datasets), las **transformaciones** y las **acciones**, que permiten manipular datos a gran escala.

También es esencial entender cómo funciona la configuración inicial, el contexto de Spark y los conceptos de **linaje** y **evaluación perezosa (lazy evaluation)**, los cuales impactan directamente en la eficiencia del procesamiento.





Elementos Básicos de Spark

Para dominar Apache Spark, es necesario comprender sus elementos fundamentales que permiten el procesamiento distribuido de datos a gran escala.

Configuración, Conexión y Contexto de Spark

Para trabajar con Spark se debe inicializar un **SparkContext**, que es el punto de entrada para cualquier operación. A través del contexto, Spark administra recursos y coordina la ejecución distribuida.

```
from pyspark import SparkContext
sc = SparkContext("local", "MiAplicacionSpark")
print(sc)
```

Componentes de la Configuración

Configuración local

local indica que el procesamiento se ejecutará en el equipo local.

Nombre de la aplicación

Útil para identificar el job en el monitor de Spark.

Driver y ejecutores

Spark coordina tareas mediante un driver y múltiples ejecutores distribuidos.

Tener correctamente configurado el contexto garantiza que las operaciones sobre datos se ejecuten de manera eficiente en clústeres grandes o entornos de prueba locales.

RDD: Qué es un RDD

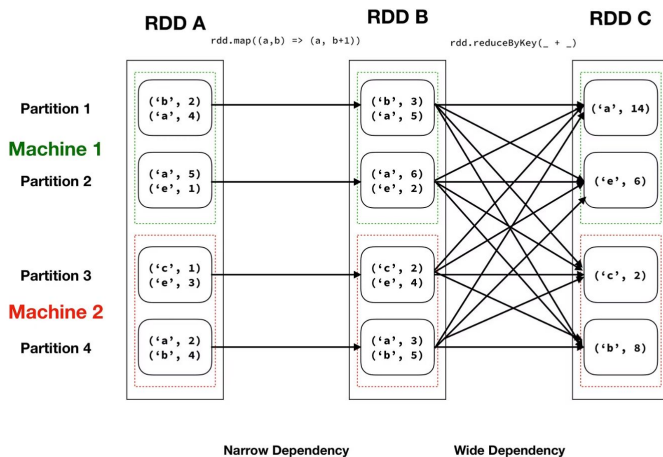
Un **RDD (Resilient Distributed Dataset)** es la estructura de datos fundamental de Spark.

Se trata de un conjunto de elementos distribuidos a través de varios nodos de un clúster, que permite:

Procesamiento paralelo: cada partición se procesa de manera independiente.

Tolerancia a fallos: puede reconstruirse a partir de su linaje si un nodo falla.

Evaluación diferida: las operaciones no se ejecutan inmediatamente, sino cuando se requiere un resultado final.



Creación y Carga de un RDD

Podemos crear un RDD de distintas maneras:

1. Desde una colección en Python

```
datos = [1, 2, 3, 4, 5]  
rdd = sc.parallelize(datos)
```

2. Desde una base de datos o fuente externa

```
rdd = sc.textFile("ruta/archivo.txt")
```

Almacenamiento de un RDD

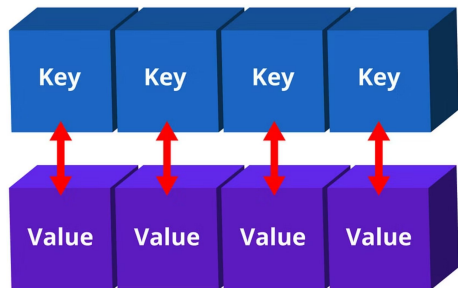
Persistencia en memoria

`rdd.persist()` mantiene los datos para múltiples usos.

Caché

`rdd.cache()` es equivalente a persistencia en memoria por defecto.

Key-Value Pairs



Pair-RDD

Un **Pair-RDD** es un RDD compuesto por pares clave-valor. Ejemplo:

```
rdd = sc.parallelize([("a", 1), ("b", 2), ("a", 3)])
```

Este tipo de RDD es fundamental para operaciones de agregación (`reduceByKey`, `groupByKey`) y es ampliamente usado en análisis de grandes volúmenes de datos.

Linaje y Lazy Evaluation

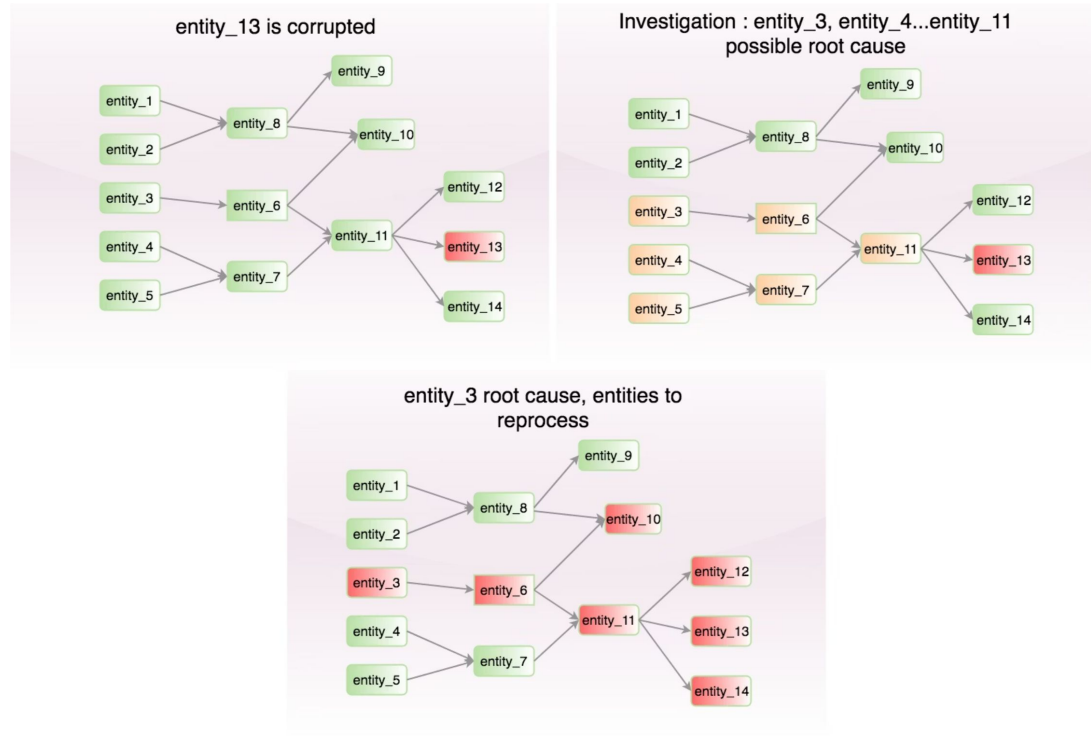
Linaje

Registro de las transformaciones aplicadas a un RDD.

Lazy Evaluation

Spark no ejecuta las transformaciones inmediatamente; espera hasta que se invoque una **acción**.

Este enfoque permite optimizaciones automáticas y evita cálculos innecesarios.



Transformaciones

Una **transformación** es una operación que crea un nuevo RDD a partir de uno existente.

Son operaciones **perezosas** (lazy), es decir, no ejecutan el cálculo hasta que se invoque una acción.

Principales Métodos de Transformación



filter(func)

Devuelve elementos que cumplen una condición.



map(func)

Aplica una función a cada elemento del RDD.



flatMap(func)

Igual que map, pero devuelve iteradores planos.



sample()

Toma una muestra aleatoria del RDD.

Más Métodos de Transformación



union()

Une dos RDD en uno solo.



distinct()

Elimina duplicados.



sortBy()

Ordena el RDD según un criterio.

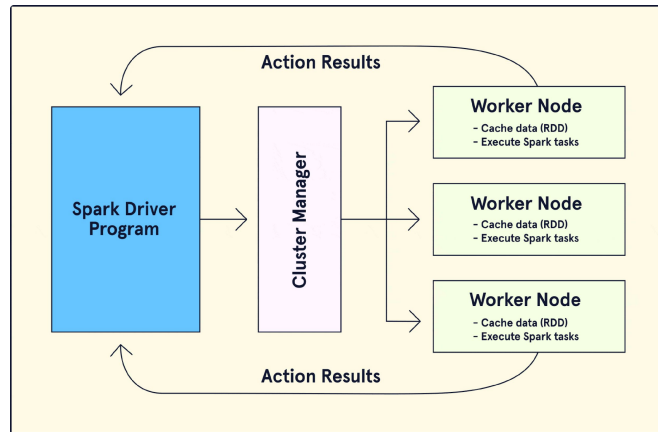
Ejemplo de Transformaciones

En este ejemplo, se crea un RDD a partir de una lista de números, se aplica una transformación map para duplicar cada valor, y luego se aplica filter para quedarse solo con los valores mayores a 10.

```
rdd = sc.parallelize([1,2,3,4,5])
resultado = rdd.filter(lambda x: x % 2 == 0).map(lambda x: x**2)
print(resultado.collect()) # [4, 16]
```

Acciones

Una **acción** ejecuta realmente las transformaciones pendientes sobre un RDD y devuelve un resultado.



Principales Acciones



collect()

Devuelve todos los elementos como una lista.



take(n)

Obtiene los primeros n elementos.



top(n)

Obtiene los n elementos más altos según el orden natural.



takeSample()

Extrae una muestra aleatoria.

Más Acciones



sum()

Suma todos los elementos
(numéricos).



mean()

Calcula la media de los elementos
(requiere RDD numérico).



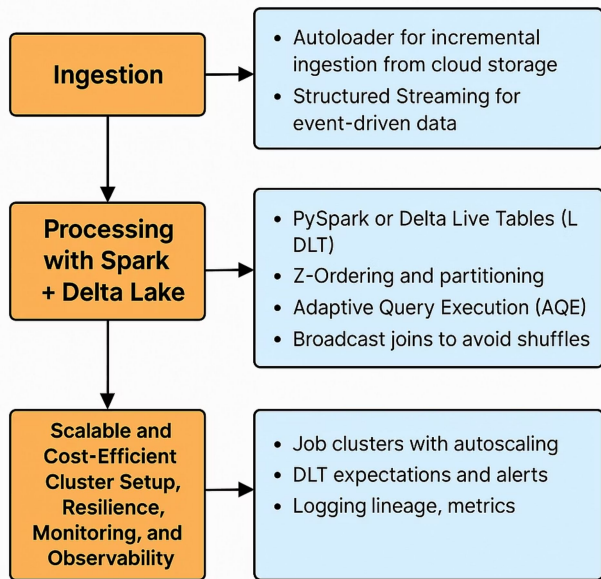
stdev()

Calcula la desviación estándar de
los datos.

Ejemplo de Acciones

En este ejemplo, se crea un RDD con una lista de números, se aplica una transformación filter para seleccionar solo los números pares, y finalmente se ejecuta la acción collect() para obtener los resultados.

```
rdd = sc.parallelize([1,2,3,4,5])  
print(rdd.sum())      # 15  
print(rdd.take(3))    # [1, 2, 3]
```



Job Spark: Qué es un Job Spark

Un **Job** en Spark es la unidad de ejecución que se lanza cuando se invoca una **acción**.

Ciclo de un Job Spark

Transformaciones registradas en DAG

Directed Acyclic Graph - Grafo

Acíclico Dirigido

División en etapas (stages)

Según dependencias entre operaciones

Ejecución en tareas distribuidas (tasks)

Realizadas por los ejecutores en el clúster

Proceso de Ejecución de un Job

Cuando llamamos a `collect()`, `count()` u otra acción, Spark:



Evalúa el linaje del RDD



Crea un plan de ejecución (DAG)



Divide el trabajo en stages y tasks



Retorna el resultado final al driver

Visualización de un Job Spark

Esta imagen muestra cómo se visualiza un Job Spark en la interfaz de usuario de Spark, con sus diferentes etapas y tareas.

Importancia de los RDDs en Spark

El manejo de **RDDs en Spark** constituye la base del procesamiento de datos distribuido.

Comprender cómo crear, almacenar y transformar RDDs, junto con la diferencia entre **transformaciones** y **acciones**, permite construir soluciones escalables y eficientes.

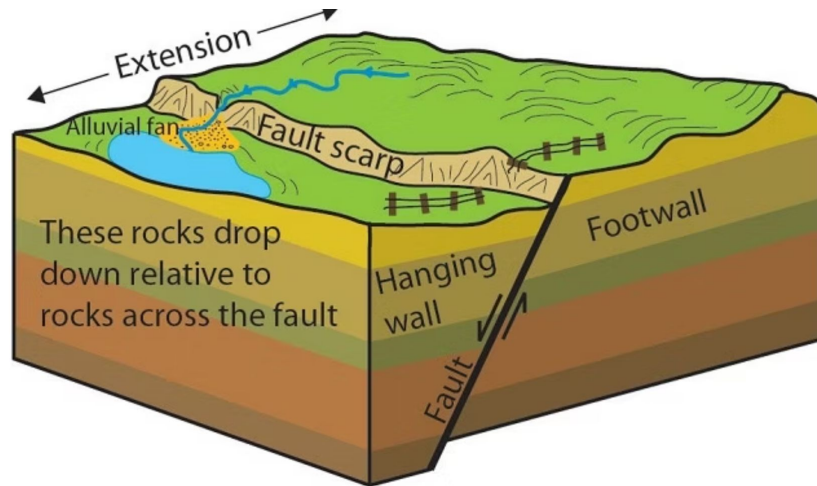
Beneficios de Lazy Evaluation y Linaje

Optimización

El concepto de **lazy evaluation** y **linaje** ofrece optimizaciones automáticas al planificar la ejecución de manera eficiente.

Tolerancia a fallos

El linaje permite reconstruir datos perdidos en caso de fallos en los nodos del clúster.



Entendiendo los Jobs de Spark

Entender cómo se ejecuta un **Job Spark** brinda claridad sobre el flujo de trabajo interno del motor.

Dominar estos elementos es fundamental para desarrollar aplicaciones que manipulen grandes volúmenes de datos de forma eficiente, cumpliendo los objetivos planteados en entornos de **Big Data**.

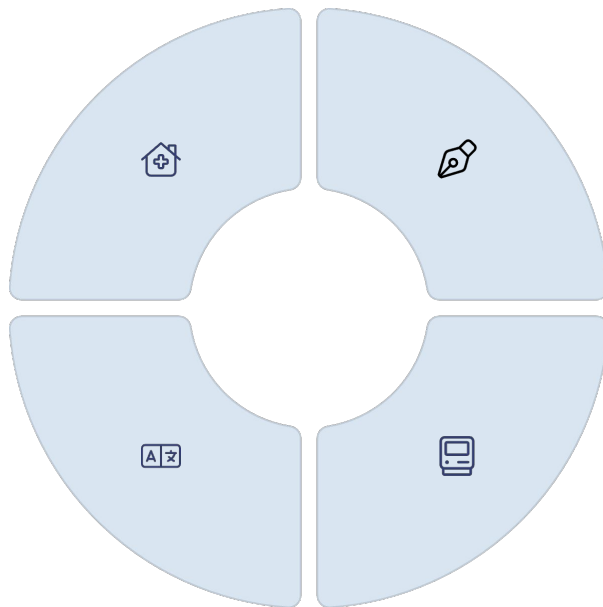
Profundizando en RDDs

Particionamiento

Los RDDs se dividen en particiones que pueden procesarse en paralelo en diferentes nodos.

Localidad

Spark intenta procesar datos cerca de donde están almacenados para reducir transferencias.



Inmutabilidad

Los RDDs no pueden modificarse una vez creados, solo transformarse en nuevos RDDs.

Computación

Cada RDD sabe cómo recalcul sus datos utilizando transformaciones anteriores.

Tipos de Persistencia en RDDs

MEMORY_ONLY

Almacena RDD como objetos Java deserializados en la JVM.

MEMORY_AND_DISK

Almacena en memoria y, si no cabe, en disco.

MEMORY_ONLY_SER

Almacena como bytes serializados (más eficiente en espacio).

DISK_ONLY

Almacena solo en disco, útil para conjuntos de datos muy grandes.

Operaciones Avanzadas con Pair-RDDs



reduceByKey(func)

Combina valores con la misma clave usando una función.



groupByKey()

Agrupar valores con la misma clave.



join(otherRDD)

Une dos RDDs por clave, similar a un JOIN en SQL.



countByKey()

Cuenta elementos por cada clave.

Ejemplo de reduceByKey

```
# Crear un Pair-RDD
pairs = sc.parallelize([("a", 1), ("b", 1), ("a", 1), ("a", 1), ("b", 1)])
# Sumar valores por claves
sums = pairs.reduceByKey(lambda a, b: a + b)
# Resultado: [("a", 3), ("b", 2)]
print(sums.collect())
```

Este ejemplo muestra cómo sumar valores asociados a la misma clave utilizando reduceByKey.



Live Coding

¿En qué consistirá la Demo?

Analizaremos paso a paso cómo manipular datos en Spark usando RDD, aplicando transformaciones y acciones para resolver un caso práctico.

1. Crear un RDD desde una colección y desde un archivo externo.
2. Aplicar transformaciones como `map()`, `filter()` y `reduceByKey()`.
3. Ejecutar acciones como `collect()` y `count()`.
4. Observar el linaje y cómo Spark optimiza con lazy evaluation.
5. Interpretar el ciclo de ejecución de un Job Spark (DAG → stages → tasks).



Momento:

Time-out!



5 -10 min.





Ejercicio

Manipulando datos con RDD en Spark

Manipulando datos con RDD en Spark

Contexto: 🙌

Vas a aplicar lo aprendido sobre RDD para manipular datos de gran tamaño de forma distribuida.

Consigna: 📝

1. Crea un RDD desde un archivo de texto creado desde 0.
2. Aplica al menos tres transformaciones y dos acciones sobre el RDD.
3. Observa el linaje y explica cómo Spark ejecutó el Job.
4. Identifica cómo el lazy evaluation impactó en el resultado y optimización.

Paso a paso: ⚙️

1. Inicializa el contexto de Spark.
2. Carga datos y crea un RDD.
3. Aplica transformaciones para filtrar y transformar los datos.
4. Ejecuta acciones para obtener métricas o resultados.
5. Documenta tu flujo y conclusiones.

Tiempo 🕒: 45 Minutos

¿Alguna consulta?



Resumen

¿Qué logramos en esta clase?

- ✓ **Comprendimos qué es un RDD y cómo se manipula en Spark.**
- ✓ **Aprendimos a aplicar transformaciones y acciones sobre datos distribuidos.**
- ✓ **Observamos el impacto de lazy evaluation y el linaje en la optimización.**
- ✓ **Vimos cómo los Jobs en Spark se dividen en Stages y Tasks.**
- ✓ **Ejercitamos la manipulación de grandes volúmenes de datos de forma distribuida y eficiente.**



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

